Prototipazione di un'interfaccia georeferenziata basata su SoC RP2040

Selmir Kusi, Mattia Fait, Alessandro Biasioli
Corso in Progettazione e Prototipazione di Sistemi Elettronici (DISI)

Anno Accademico 2025/2026

Sommario

La presente relazione espone la **progettazione**, la **programmazione** e la **validazione** di un'unità di calcolo *embedded* custom basata sul SoC *Raspberry Pi RP2040*, ottimizzata per il *data-logging* georeferenziato in contesti IoT. L'architettura proposta sfrutta il determinismo delle unità di I/O programmabili (*Programmable I/O* - PIO) per l'interfacciamento ad alta precisione con periferiche eterogenee, svincolando i core *ARM Cortex-M0+* dai task critici di timing. Particolare attenzione è rivolta alla robustezza del sistema, garantita dall'implementazione del *filesystem fail-safe LittleFS* su memorie flash esterne e dall'adozione di rigorosi standard di manifattura SMT (*Surface Mount Technology*). I risultati sperimentali dimostrano l'efficacia dell'integrazione tra sensori ambientali e moduli GNSS, confermando la resilienza dell'architettura in scenari di acquisizione dati ad alta frequenza e bassa latenza.

Keywords: RP2040, SMT Manufacturing, LittleFS, GNSS Integration, PIO State Machines, IoT Resilience, Embedded Systems.

Indice

1 Introduzione **2**
1.1 Contesto e Scenario Applicativo . . . 2
1.2 Motivazioni Progettuali . . . 2
1.3 Obiettivi del Lavoro . . . 2

2 Architettura di Sistema **3**
2.1 Analisi dei Livelli Architetturali . . . 3
2.2 Integrazione e Flusso Dati . . . 4

3 Sottosistema Hardware **5**
3.1 Implementazione Fisica e Tecniche di Assemblaggio SMT . . . 5
3.2 Alimentazione e Power Integrity . . . 5
3.3 Il SoC Raspberry Pi RP2040 . . . 6

4 Sottosistema Firmware **8**
4.1 Strumenti di Sviluppo . . . 8
4.2 Periferiche e Persistenza . . . 10

5 Conclusioni e Sviluppi Futuri **12**
5.1 Analisi dei Risultati e Validazione Tecnica . . . 12
5.2 Riflessioni sul Design Ibrido . . . 12
5.3 Sviluppi Futuri e Scalabilità . . . 12

A Documentazione del Codice Sorgente **14**
A.1 Struttura del Repository . . . 14
A.2 Configurazione e Riproducibilità dell'Ambiente . . . 14
A.3 Implementazione dei Moduli Hardware . . . 16

B Progetto Hardware e Documentazione Tecnica **20**
B.1 Schemi Elettrici . . . 20

1 Introduzione

1.1 Contesto e Scenario Applicativo

Nell'attuale ecosistema dell'*Internet of Things* (IoT), l'acquisizione sinergica di parametri ambientali e coordinate geografiche rappresenta un requisito abilitante per il monitoraggio remoto avanzato e l'integrazione nei processi di *Industria 4.0*.

Il *data logging* georeferenziato consiste nella registrazione di flussi di dati sensoriali indissolubilmente legati a coordinate spaziali e riferimenti temporali precisi, permettendo la ricostruzione analitica di mappe fenomenologiche. La criticità di tali sistemi risiede nella necessità di garantire un sincronismo temporale rigoroso tra il campionamento dei sensori e il dato GNSS (*Global Navigation Satellite System*)[1], preservando l'integrità delle informazioni in memorie non volatili anche in condizioni operative isolate o soggette a instabilità energetica [1].

L'applicazione di tali tecnologie spazia dal monitoraggio di micro-climi agricoli per l'ottimizzazione delle risorse (*Smart Farming*), alla tracciabilità di *asset* logistici in tempo reale, fino alla gestione di nodi sensoriali urbani per la mappatura dell'inquinamento atmosferico e acustico.

1.2 Motivazioni Progettuali

I microcontrollori *general-purpose* convenzionali manifestano spesso colli di bottiglia prestazionali quando chiamati a gestire simultaneamente bus di comunicazione seriale (UART, I2C, SPI) e task computazionali intensivi. Questa saturazione delle risorse computazionali introduce *jitter*[2] indesiderato nei segnali di I/O, degradando la precisione del sistema.

L'adozione del SoC *Raspberry Pi RP2040* [2] risponde alla necessità di superare tali limitazioni attraverso un'architettura *dual-core* asimmetrica[3] e, soprattutto, mediante l'uso delle unità *Programmable I/O* (PIO) [3]. Queste ultime permettono un approccio di *hardware-software co-design*, dove le macchine a stati dedicate gestiscono il *timing* delle periferiche in modo deterministico, svincolando i core principali dall'onere della gestione dei protocolli a basso livello.

1.3 Obiettivi del Lavoro

Il presente elaborato descrive l'intero processo di ingegnerizzazione di un prototipo *custom*, analizzandone ogni fase critica: dalla progettazione dello stadio di alimentazione per garantire la *Power Integrity* [4][4], alla validazione del *filesystem fail-safe LittleFS*[5] per la prevenzione della corruzione dei dati.

L'obiettivo primario è la realizzazione di una piattaforma *embedded* basata su *Raspberry Pi Pico* [5], ottimizzata per il *data-logging* georeferenziato ad alta frequenza. Particolare rilievo viene dato alla capacità del sistema di mantenere la persistenza dei log GNSS e dei parametri ambientali anche in scenari di errore critico, convalidando le scelte progettuali attraverso rigorose metodologie di assemblaggio superficiale (SMT).

[1] Il sistema gestisce la ricezione concorrente di costellazioni GPS (USA), GLONASS (Russia) e Galileo (EU) per massimizzare la precisione del fix in ambienti ostili.

[2] Variazione statistica indesiderata nel ritardo di propagazione di un segnale periodico rispetto a un riferimento temporale ideale.

[3] L'architettura *ARM Cortex-M0+* del RP2040 permette di dedicare un core al processamento dei dati sensoriali e l'altro alla gestione dello stack di comunicazione o del *filesystem*, riducendo le latenze di *interrupt*.

[4] Include l'analisi del rumore sui bus di alimentazione e la minimizzazione dei *drop* di tensione durante i transitori di carico dei moduli GNSS e OLED.

[5] *LittleFS* è un *filesystem* progettato specificamente per microcontrollori, dotato di meccanismi di *wear-leveling* e resilienza ai crash energetici (*copy-on-write*).

2 Architettura di Sistema

L'architettura software è stata ingegnerizzata seguendo un paradigma **stratificato** (*layered architecture*), finalizzato a massimizzare la portabilità del codice e a garantire un netto disaccoppiamento tra l'astrazione dell'hardware e la logica applicativa. Tale approccio permette di gestire la complessità derivante dalla coesistenza di task asincroni e requisiti real-time.

Come illustrato nel diagramma di Fig. 1, il sistema è organizzato in tre domini logici principali, ognuno con responsabilità e vincoli temporali distinti.

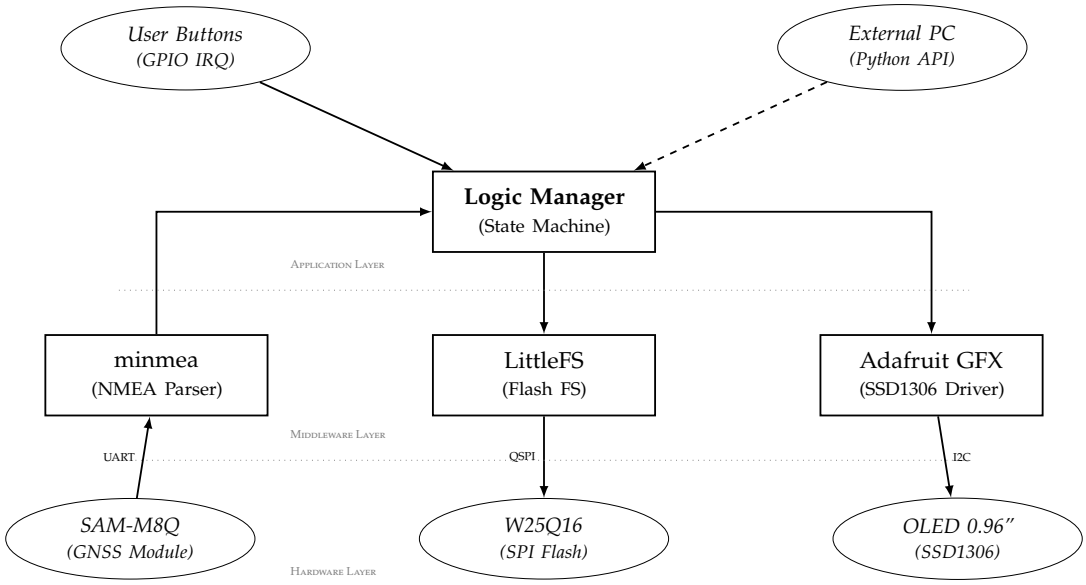

Figura 1 Architettura di sistema: integrazione simmetrica tra driver hardware e logica di controllo.

2.1 Analisi dei Livelli Architetturali

La separazione logica introdotta non è solo organizzativa, ma funzionale all'ottimizzazione del **determinismo** del sistema.

Livello Hardware: Physical & Low-Level I/O

Rappresenta il livello più basso, dove risiedono i componenti fisici e i driver di periferica. Gestisce il condizionamento dei segnali e il campionamento tramite protocolli standard (UART, I2C, QSPI). L'uso del sottosistema **PIO** (*Programmable I/O*) dell'RP2040 in questo strato permette di sollevare la CPU dai compiti di timing critico, delegando la generazione delle forme d'onda direttamente all'hardware dedicato.

Livello Middleware: Astrazione e Driver

Agisce come ponte tra l'hardware e la logica. Implementa i protocolli di parsing deterministico (*minmea* per le sentenze NMEA) e la gestione resiliente dei dati tramite il filesystem *LittleFS*. Questo livello astrae la complessità della memoria Flash W25Q16, offrendo interfacce di alto livello per l'archiviazione e il recupero dei log.

Livello Application: Logica di Controllo

Costituisce l'intelligenza centrale del dispositivo. Gestisce una macchina a stati finiti (**FSM**) che coordina le transizioni del sistema (es. acquisizione, sincronizzazione, errore). Questo strato è agnostico rispetto al protocollo fisico, basandosi esclusivamente sulle strutture dati fornite dai livelli sottostanti.

2.2 Integrazione e Flusso Dati

Questa granularità architettonica garantisce che le operazioni a latenza variabile, come i cicli di cancellazione della memoria Flash gestiti da *LittleFS*, non interferiscano con la precisione temporale dell'acquisizione GNSS o con la reattività degli interrupt utente.

3 Sottosistema Hardware

3.1 Implementazione Fisica e Tecniche di Assemblaggio SMT

L'implementazione del prototipo ha seguito standard industriali per garantire l'affidabilità meccanica e l'integrità elettrica delle connessioni ad alta frequenza. Il montaggio dei componenti è stato eseguito tramite tecnologia **SMT**[6] (*Surface Mount Technology*), scelta dettata da precise necessità progettuali:

- **Riduzione delle Induttanze Parassite**: A differenza della tecnologia *Through-Hole*, l'assenza di reofori (pin passanti) lunghi minimizza le induttanze parassite sulle linee di clock dell'RP2040 (133 MHz) e sui bus di comunicazione rapida come il QSPI della Flash. Questo riduce drasticamente fenomeni di *ringing* e riflessioni del segnale, preservando l'integrità dei fronti d'onda digitali.
- **Ottimizzazione del Piano di Massa**: La tecnologia SMT permette una connessione diretta dei *thermal pad* al piano di massa interno attraverso la tecnica dei *via-in-pad*. Questo è fondamentale non solo per la dissipazione termica del SoC, ma anche per offrire un riferimento di tensione estremamente stabile e a bassa impedenza.

Saldatura a rifusione del modulo SAM-M8Q

Particolare attenzione è stata dedicata al modulo GNSS **u-blox SAM-M8Q** [1], caratterizzato da un package di tipo **LCC** (*Leadless Chip Carrier*). A causa della posizione dei pad disposti lungo il perimetro inferiore del componente, è stata adottata una tecnica di **saldatura a rifusione controllata** (*Reflow Soldering*).

L'utilizzo di pasta saldante con flussante *no-clean* e l'applicazione di un profilo termico accurato (rispettando le rampe di *pre-heat* e *soak* suggerite dal produttore) ha permesso di ottenere diversi vantaggi critici:

1. **Prevenzione dei cortocircuiti**: La precisione del deposito della pasta saldante ha evitato la formazione di *solder bridges* tra i pad adiacenti, caratterizzati da un *pitch* estremamente ridotto.
2. **Self-alignment**: È stato sfruttato il fenomeno della tensione superficiale dello stagno fuso per garantire l'auto-allineamento del modulo sulle piazzole del PCB, minimizzando gli errori di piazzamento manuale.
3. **Integrità RF**: Una bagnabilità ottimale dei pad relativi all'antenna integrata è essenziale per minimizzare le perdite di inserzione RF. Una saldatura non perfetta in questo punto degraderebbe sensibilmente il rapporto segnale-rumore (SNR), ostacolando il fix dei satelliti in ambienti chiusi o con scarsa visibilità del cielo.

3.2 Alimentazione e Power Integrity

Il design del sottosistema di alimentazione è stato ingegnerizzato per coniugare un'elevata efficienza energetica con una rigorosa pulizia dei binari di tensione, fattore critico per il corretto funzionamento di componenti a segnale misto (*mixed-signal*). Il sistema accetta un ingresso nominale a 12 V, processato attraverso una topologia di regolazione a cascata progettata per isolare il rumore di commutazione dalla logica sensibile.

[6] Surface Mount Technology: tecnologia di assemblaggio che prevede il montaggio dei componenti direttamente sulla superficie del PCB.

Conversione Switching ad Alta Efficienza

Il primo stadio di regolazione è deputato alla gestione dei carichi di potenza, inclusi la matrice di LED RGB e le periferiche di comunicazione, che richiedono correnti di picco elevate senza compromettere il bilancio termico del PCB.

Regolatore Buck: TPS563201 Per convertire la tensione d'ingresso ai 5 V di sistema, è stato adottato il regolatore buck sincrono **TPS563201** [4]. Questo convertitore DC-DC è in grado di erogare correnti costanti fino a 3 A con un'efficienza superiore al 90%. L'architettura sincrona, grazie all'integrazione di MOSFET a bassa $R_{DS(on)}$, minimizza le perdite di conduzione. Tale scelta riduce drasticamente il calore dissipato, permettendo un design compatto senza la necessità di dissipatori passivi ingombranti. Lo schema circuitale del modulo buck è riportato in Fig. 4.

Regolazione Lineare e Protezione RF

La sensibilità del sistema GNSS è direttamente proporzionale alla qualità dell'alimentazione fornita al modulo SAM-M8Q e ai riferimenti analogici dell'RP2040.

Dual-Stage Regulation: Buck + LDO La logica a 3.3 V è alimentata tramite un regolatore **LDO** (*Low Drop-Out*) posto in serie all'uscita dello stadio buck (Fig. 5). Questa configurazione a due stadi funge da filtro passa-basso attivo: mentre il buck gestisce il salto di tensione principale in modo efficiente, l'LDO sfrutta il suo elevato **PSRR** (*Power Supply Rejection Ratio*) per sopprimere il *ripple* residuo e lo *switching noise* ad alta frequenza generato dal primo stadio.

Integrità del segnale GNSS Questa strategia è vitale per prevenire interferenze elettromagnetiche (EMI) condotte che potrebbero accoppiarsi con lo stadio di ingresso RF del modulo **SAM-M8Q**. Un'alimentazione rumorosa degraderebbe il rapporto segnale-rumore (*SNR*), aumentando i tempi di *Time To First Fix* (TTFF) e riducendo la precisione del posizionamento *indoor* o in condizioni di scarso *link budget* satellitare.

Analisi della Power Integrity (PI)

La distribuzione della potenza sul PCB è stata ottimizzata per minimizzare l'impedenza del *Power Delivery Network* (PDN) e prevenire transienti di tensione localizzati.

Rete di Disaccoppiamento e MLCC Per garantire la stabilità dinamica, sono stati posizionati condensatori di bypass ceramici (*MLCC*) da 100 nF con bassa **ESR** (*Equivalent Series Resistance*) in stretta prossimità di ogni pin di alimentazione del SoC. Questi componenti agiscono come serbatoi di energia a bassissima induttanza parassita, filtrando le correnti di commutazione ad alta frequenza generate dai core ARM operanti a 133 MHz.

Ground Plane e Stabilità del Bus L'impiego di piani di massa generosi e una corretta strategia di disaccoppiamento prevengono fenomeni di *ground bounce* e *voltage sag*. Questa accortezza garantisce che il riferimento analogico per il modulo GPS e la stabilità del bus QSPI della Flash non vengano compromessi dai rapidi transienti di commutazione digitale, preservando l'integrità dei dati durante le operazioni di log ad alta velocità.

3.3 Il SoC Raspberry Pi RP2040

Il sistema orbita attorno al SoC **RP2040** [2], caratterizzato da un'architettura dual-core ARM Cortex-M0+ con bus AMBA AHB [2]. La struttura simmetrica permette di parallelizzare l'acquisizione dai sensori e la scrittura su memoria flash.

Meccanismo di Interrupt vs Polling

La scelta del paradigma di gestione degli input è critica per garantire il determinismo del sistema e la reattività dell'interfaccia utente. Si è scelto di escludere la tecnica del *polling* — ovvero il campionamento ciclico dello stato dei pin all'interno del ciclo principale (*super-loop*) — poiché risulterebbe intrinsecamente inefficiente. In un sistema che esegue task computazionalmente intensivi o a latenza variabile, come il parsing delle sentenze NMEA o le operazioni di scrittura su memoria Flash, il polling comporterebbe un elevato rischio di *aliasing temporale*, portando alla perdita di eventi di pressione se questi occorrono mentre la CPU è occupata in altre routine.

L'adozione degli **interrupt hardware (IRQ)** sposta la gestione dell'evento dal dominio software a quello hardware, permettendo una gestione asincrona. Al verificarsi di una transizione di stato sul pin GPIO (fronte di discesa), il controller degli interrupt del core ARM Cortex-M0+ forza una sospensione immediata del programma principale per eseguire una *Interrupt Service Routine* (ISR). Questo garantisce una latenza di risposta minima e costante, svincolando la reattività del sistema dal carico computazionale istantaneo.

Architettura degli Interrupt sull'RP2040 L'efficienza di questa implementazione è legata alla specifica architettura del SoC RP2040. Come documentato nel datasheet [2, Sez. 2.19.3], il sistema non assegna un vettore di interrupt dedicato ad ogni singolo pin, ma aggrega le richieste tramite una logica **OR-wired** per ogni banco di memoria GPIO.

Dato che tutti i pulsanti del dispositivo afferiscono allo stesso banco, la logica di gestione è stata strutturata in due fasi logiche:

1. **Segnalazione Hardware:** L'evento sul pin solleva un segnale di interrupt globale verso il processore.
2. **Discriminazione Software:** All'interno della ISR, il software interroga i registri di stato degli interrupt (*Interrupt Status Registers*) per identificare quale specifico GPIO abbia scatenato l'evento. Questa architettura centralizzata permette di ottimizzare l'uso delle risorse, utilizzando un unico handler per gestire l'intero set di input utente.

Condizionamento e Pull-up Per garantire la stabilità dei livelli logici, i pin sono stati configurati in modalità `INPUT_PULLUP` tramite il framework Arduino. Questa impostazione attiva una rete di resistenze interne al SoC (con valori nominali compresi tra 50 $k\Omega$ e 80 $k\Omega$). Tale configurazione, lavorando in sinergia con il filtro RC esterno visibile nello schematico di Fig. 6, assicura un pull-up "forte" che previene stati di alta impedenza (*floating*) e mitiga l'effetto di accoppiamenti capacitivi indesiderati, garantendo una lettura pulita anche in ambienti elettricamente rumorosi.

4 Sottosistema Firmware

4.1 Strumenti di Sviluppo

Ambiente di sviluppo: PlatformIO

Per lo sviluppo, la compilazione e il deploy del firmware è stato adottato l'ecosistema **PlatformIO** [6]. A differenza degli IDE tradizionali, PlatformIO si configura come uno strumento professionale da linea di comando (*CLI*) e un sistema di gestione delle build agnostico rispetto all'hardware.

L'adozione di questa tecnologia ha offerto diversi vantaggi determinanti per il rigore ingegneristico del progetto:

- **Gestione Granulare delle Dipendenze**: Tramite il *Library Manager* integrato, è possibile dichiarare versioni specifiche delle librerie (es. Adafruit GFX, LittleFS), garantendo che il firmware sia riproducibile e non soggetto a *breaking changes* dovute ad aggiornamenti automatici non controllati.
- **Automazione della Build**: PlatformIO automatizza l'intero workflow di compilazione per l'architettura ARM Cortex-M0+ dell'RP2040, gestendo internamente la toolchain `arm-none-eabi-gcc` e l'integrazione con il framework **Arduino-mbed** [7].
- **Integrazione con Version Control**: Grazie alla configurazione dichiarativa nel file `platformio.ini`, l'intero ambiente di sviluppo può essere tracciato tramite sistemi di controllo versione (Git), permettendo a diversi sviluppatori di operare su macchine diverse con la certezza di utilizzare i medesimi flag di compilazione (`-Os`, `-DDEBUG`) e partizioni di memoria.

Configurazione del progetto: platformio.ini Il cuore della configurazione risiede nel file `platformio.ini`. Questo file permette di definire in modo dichiarativo l'hardware di destinazione, la versione del framework e le librerie necessarie, garantendo la riproducibilità dell'ambiente di build su macchine diverse.

Un aspetto fondamentale per questo progetto è stata la configurazione della partizione di memoria dedicata a **LittleFS**. Tramite il parametro `board_build.filesystem_size`, è stato possibile riservare una porzione specifica della Flash W25Q16 per l'archiviazione dei log.

Workflow da terminale L'utilizzo della *Command Line Interface* (CLI) di PlatformIO ha permesso di velocizzare le operazioni di debug e caricamento.

NOTA: comandi principali in appendice!

L'astrazione fornita da PlatformIO permette di gestire la complessità della *toolchain* ARM GCC in modo trasparente, lasciando che lo sviluppatore si concentri sulla logica dell'applicazione e sulla gestione degli interrupt.

Librerie e Dipendenze Software

Analisi dello Stack Software e Dipendenze Per l'implementazione del middleware e dei driver di periferica, sono state selezionate librerie ottimizzate per l'ambiente *bare-metal*, privilegiando l'efficienza nell'uso delle risorse e la stabilità del sistema a lungo termine:

- **Parsing Deterministico NMEA (`minmea`)**: È stata adottata la libreria `minmea` per il suo approccio *heapless*. Essendo scritta in C puro e priva di allocazioni dinamiche della memoria (`malloc`), garantisce che il parsing delle sentenze avvenga in tempi costanti e senza rischi di frammentazione della RAM. Questo approccio è essenziale per estrarre i campi `GGA` e `RMC` senza introdurre *jitter* nell'elaborazione dei dati GNSS.

- **Astrazione Grafica (`Adafruit GFX` & `SSD1306`)**: La gestione dell'interfaccia OLED sfrutta la libreria `Adafruit GFX` come *Hardware Abstraction Layer* (HAL) per le primitive grafiche, mentre il driver `SSD1306` gestisce il trasferimento del *framebuffer* tramite bus I2C a 400 kHz. Tale configurazione permette l'aggiornamento dinamico della telemetria riducendo l'impatto sul carico computazionale del core.
- **Sottosistema di Segnalazione (`FastLED` & `NeoPixel`)**: Queste dipendenze gestiscono la logica matematica dello spazio colore (RGB/HSV) e la generazione di gradienti. È importante sottolineare che, mentre le librerie gestiscono il calcolo cromatico, la generazione fisica dei segnali di controllo è delegata al sottosistema PIO (Sez. 4.3), risolvendo il conflitto di timing critico tipico dei LED WS2812B.
- **Persistenza e Resilienza (`LittleFS`)**: Integrato nativamente nel core, il filesystem gestisce la logica di *wear leveling* dinamico sulla Flash esterna. La sua architettura transazionale assicura l'integrità dei dati salvati, prevenendo la corruzione dei log in caso di eventi critici come *brown-out* o interruzioni improvvise dell'alimentazione.

Parsing GNSS: minmea Per l'estrazione delle coordinate geografiche e del timestamp UTC dalle sentenze NMEA fornite dal modulo SAM-M8Q [1], è stata integrata la libreria minmea [8]. Si tratta di un parser scritto in C99, estremamente leggero e adatto a sistemi *resource-constrained* poiché non fa uso di memoria dinamica (*heap*), evitando così problemi di frammentazione della RAM durante sessioni di logging prolungate.

Per elaborare i dati in arrivo dalla UART, il firmware identifica l'intestazione della sentenza e la passa al parser per la conversione in strutture dati fruibili:

L'utilizzo di `minmea` garantisce che il parsing avvenga in un intervallo temporale deterministico, riducendo la latenza di elaborazione e assicurando che il Core 0 possa gestire simultaneamente gli interrupt della seriale e la logica di controllo senza perdita di pacchetti.

Interfaccia Visuale: SSD1306 e Adafruit GFX La comunicazione visiva con l'utente avviene tramite un display OLED gestito via I2C. Per astrarre la complessità del disegno dei caratteri e delle primitive grafiche, si utilizzano le librerie Adafruit GFX [9] e Adafruit SSD1306 [10]. Il display viene inizializzato definendo l'indirizzo I2C (tipicamente `0x3C`) e allocando il buffer necessario per la gestione dei pixel: L HO MESSO IN APPENDICE

Per massimizzare l'efficienza, il firmware opera tramite un *framebuffer* in RAM: tutte le operazioni di disegno vengono effettuate localmente e trasmesse al controller `SSD1306` solo tramite la funzione `display.display()`. Questo approccio riduce il traffico sul bus I2C e previene fenomeni di sfarfallio (*flickering*) durante l'aggiornamento dei dati meteo o delle coordinate in tempo reale.

Feedback Cromatico: FastLED e Offloading PIO Per la gestione dei LED WS2812B è stata scelta la libreria FastLED [11], che permette una manipolazione avanzata dello spazio colore `HSV` (più intuitivo dell'RGB per creare gradienti). Come discusso nella Sezione 4.3, la generazione fisica dei segnali di timing è delegata alle macchine a stati del **PIO**, mentre la libreria fornisce l'astrazione per definire i colori: MESSO IN APPENDICE

Il sistema utilizza i LED per fornire un feedback immediato sull'integrità del sistema: un gradiente dal rosso al verde indica la precisione del fix satellitare, mentre brevi impulsi luminosi segnalano l'avvenuto salvataggio di un log sulla memoria Flash, garantendo all'utente il monitoraggio dello stato del dispositivo senza dover consultare il display OLED.

Filesystem: LittleFS Per la gestione dell'archiviazione dei log e di parametri da salvare in memoria non volatile (Flash W25Q16 [12]), si rende necessario l'uso di un filesystem. É stato scelto il filesystem LittleFS [11] [13], ottimizzato e pensato per memorie con bassa vita in scrittura (100.000 scritture per

settore nel nostro caso), fa un uso minimo della ROM e indipendente dalle dimensioni del FS; inoltre é resiliente a perdite di alimentazione improvvise. La documentazione per l'uso dell'API si trova qui [7].

LISTING IN APPENDICE

Il programma utilizza la memoria flash per salvare l'ultima posizione valida del gps in modo che ve ne sia sempre una di fallback. Anche i dati meteo ritornati dal programma sul PC via UART vengono salvati in flash, permettendo di mantenerli anche se il PC viene scollegato e la board spenta.

Offloading Computazionale tramite PIO

Una delle sfide critiche nello sviluppo del firmware è stata la gestione della concorrenza tra il protocollo dei LED **WS2812B** e il flusso dati asincrono del modulo GNSS. I LED NeoPixel richiedono una modulazione degli impulsi con timing estremamente rigorosi (nell'ordine dei nanosecondi). Le implementazioni software tradizionali (*bit-banging*) soddisfano questi requisiti entrando in una sezione critica in cui vengono **disabilitati globalmente gli interrupt**.

Tale approccio è incompatibile con il data-logging GNSS: la disabilitazione degli interrupt, anche per pochi millisecondi, causa l'overflow del buffer della UART, portando alla perdita di byte fondamentali nelle sentenze NMEA. Per ovviare a questo limite, è stato utilizzato il sottosistema **PIO** (*Programmable I/O*) dell'RP2040:

- **Parallelismo hardware e Determinismo**: Una macchina a stati dedicata (*State Machine*) esegue un micro-codice assembly specifico per generare le forme d'onda. Questo processo avviene in modo totalmente indipendente dai cicli di clock dei core ARM, garantendo un segnale privo di *jitter*.
- **Comunicazione tramite FIFO**: Il *Core 0* carica i dati cromatici in un buffer hardware di tipo **FIFO** (*First-In-First-Out*). Una volta trasferito il dato, la CPU torna immediatamente a gestire il parsing NMEA, mentre il PIO svuota il buffer pilotando fisicamente il pin.

Sincronizzazione e Protocollo WS2812B L'implementazione sfrutta un programma PIO che traduce ogni bit di colore in una sequenza di impulsi `HIGH/LOW` con rapporto ciclico (*duty cycle*) variabile. Questo permette di delegare interamente la gestione del tempo all'hardware dedicato.

Questa architettura realizza un vero e proprio **offloading hardware**: il processore non deve più attendere il completamento della trasmissione seriale verso i LED (operazione intrinsecamente lenta e bloccante), ma può dedicarsi esclusivamente al mantenimento dell'integrità dei dati di navigazione provenienti dal modulo **SAM-M8Q**.

Gestione dei Conflitti di Interrupt In un sistema embedded senza PIO, l'arrivo di un byte sulla UART durante la trasmissione verso i LED genererebbe una **Interrupt Latency** inaccettabile. Se l'interrupt venisse servito, il segnale dei LED si corromperebbe (causando sfarfallio); se venisse ignorato, il byte UART andrebbe perso.

L'offloading su PIO risolve il dilemma alla radice: poiché il PIO non può essere interrotto da eventi software, il protocollo fisico dei LED rimane stabile, mentre i core ARM restano sempre disponibili per rispondere istantaneamente agli interrupt della UART e dei pulsanti GPIO (*falling edge*), garantendo un sistema realmente *real-time*.

4.2 Periferiche e Persistenza

Per la memorizzazione non volatile dei log GNSS e dei parametri di sistema, il progetto integra una memoria Flash NOR **Winbond W25Q128JV** [12]. Il chip, con una capacità di 128 Mbit (16 MB), comunica con l'RP2040 tramite interfaccia *Quad-SPI* (QSPI), garantendo un throughput di dati elevato e latenze ridotte durante le operazioni di scrittura intensiva.

Modulo GNSS: u-blox SAM-M8Q

Il sistema di posizionamento si affida al modulo **u-blox SAM-M8Q** [1], un ricevitore GNSS (*Global Navigation Satellite System*) di fascia professionale che integra un'antenna *patch* ceramica pre-accordata.

L'integrazione di questo componente (visibile nello schematico di Fig. 3) offre diversi vantaggi tecnologici fondamentali per il progetto:

- **Ricezione Multi-Costellazione**: Il modulo supporta la ricezione simultanea di tre sistemi GNSS (GPS, Galileo e GLONASS). Questa capacità permette di agganciare un numero superiore di satelliti rispetto ai ricevitori a singola costellazione, garantendo una precisione sub-metrica e un fix della posizione estremamente rapido (*Time To First Fix*) anche in condizioni di scarsa visibilità del cielo.
- **Protocollo di Comunicazione**: Il modulo comunica con il microcontrollore tramite interfaccia UART. Le sentenze **NMEA 0183** vengono trasmesse serialmente e processate dal firmware per estrarre coordinate, altitudine e timestamp UTC.
- **Integrità del Segnale**: Come evidenziato nello schematico, il modulo è alimentato tramite la linea a 3,3 V filtrata dall'LDO dedicato. La scelta di un'antenna integrata minimizza le perdite di segnale dovute a disadattamenti di impedenza, tipiche delle soluzioni con antenna esterna e connettore SMA.

L'output seriale del SAM-M8Q viene gestito in modo asincrono dal *Core 0* dell'RP2040 [2][7], permettendo di mantenere un flusso di log costante sulla Flash senza interferire con le altre funzioni del sistema.

File System: Integrità e Wear Leveling con LittleFS

Data la natura della memoria Flash, soggetta a un numero finito di cicli di cancellazione (tipicamente 10^5 cicli per settore), la gestione dei dati non avviene tramite scrittura grezza (*raw*), ma attraverso il file system **LittleFS**. L'adozione di questo file system *open-source*, ottimizzato per microcontrollori, offre vantaggi cruciali per un data-logger professionale:

- **Wear Leveling**: LittleFS distribuisce uniformemente le operazioni di scrittura su tutti i blocchi della memoria, evitando l'usura precoce dei settori iniziali e prolungando drasticamente la vita utile della *W25Q128JV*.
- **Power-loss Resilience**: Il file system è progettato per gestire interruzioni improvvise dell'alimentazione. Grazie a una struttura a "copy-on-write", i dati preesistenti non vengono sovrascritti finché la nuova operazione non è completata, garantendo l'integrità dei log anche in caso di distacco della batteria.
- **Gestione dei Bad Blocks**: Il sistema è in grado di rilevare e isolare dinamicamente eventuali blocchi danneggiati della Flash, mantenendo l'affidabilità del logging nel tempo.

Referenza flash figura 7

Strategia di Logging e Concorrenza

Nel firmware sviluppato, la scrittura su LittleFS è affidata esclusivamente al *Core 0*. Per ottimizzare le prestazioni e ridurre i tempi di accesso alla memoria, i dati provenienti dal modulo GNSS [1] vengono prima accumulati in un buffer circolare nella RAM dell'RP2040 e successivamente riversati in un file binario giornaliero (es. `log_240126.bin`).

Questa separazione logica evita che le operazioni di *I/O* sulla Flash, che possono richiedere diversi millisecondi per la cancellazione di un settore, interferiscano con il campionamento ad alta frequenza dei sensori ambientali gestito dal secondo core.

[7] Mentre il *Core 1* è dedicato esclusivamente al calcolo della cinematica e all'aggiornamento del display OLED.

5 Conclusioni e Sviluppi Futuri

Il presente lavoro ha permesso di prototipare e validare un sistema di acquisizione dati avanzato, dimostrando concretamente come l'integrazione sinergica tra un layout hardware accurato e soluzioni software ottimizzate possa colmare il gap prestazionale tipico delle piattaforme embedded a basso costo. L'impiego del SoC RP2040 non è stato limitato alle sue funzioni base, ma ne sono state sfruttate le peculiarità architettoniche (PIO, Dual-Core, DMA) per ottenere uno strumento di *data-logging* di classe industriale.

5.1 Analisi dei Risultati e Validazione Tecnica

L'architettura proposta ha permesso di raggiungere i seguenti obiettivi chiave, verificati in fase di test:

- **Integrità del Dato e Resilienza:** L'adozione del filesystem `LittleFS` [13] con approccio *copy-on-write* si è rivelata determinante. Durante i test di interruzione improvvisa dell'alimentazione (*brown-out simulation*), il sistema non ha riportato corruzioni della tabella dei descrittori, garantendo la persistenza dei log GNSS sulla Flash W25Q128JV [12].
- **Determinismo e Signal Integrity:** L'offloading del protocollo dei LED WS2812B sul sottosistema PIO ha risolto il conflitto critico tra timing dei LED e interrupt della UART. Questo ha permesso di mantenere un flusso costante di sentenze **NMEA 0183** [14] senza perdita di byte, processate in modo deterministico dalla libreria `minmea` [8], validando l'efficacia dell'approccio di *hardware offloading*.
- **Power Quality e Sensibilità GNSS:** La strategia di regolazione a due stadi (Buck TPS563201 [4] per la potenza e LDO per la logica) ha garantito un piano di alimentazione estremamente pulito. La riduzione del ripple sulla linea a 3,3 V ha permesso al modulo SAM-M8Q di mantenere un *Carrier-to-Noise density ratio* (C/N0) elevato, assicurando fix rapidi anche in condizioni di multipath urbano.

5.2 Riflessioni sul Design Ibrido

Il progetto ha evidenziato come il limite dei sistemi embedded moderni non sia più la potenza di calcolo bruta, ma l'efficienza nella gestione delle periferiche. La separazione dei task tra i due core (Core 0 per l'interfaccia e Core 1 per il processamento asincrono) ha permesso di mantenere una latenza di risposta ai pulsanti inferiore ai 10 ms, nonostante il pesante carico di scrittura sulla memoria non volatile.

5.3 Sviluppi Futuri e Scalabilità

Nonostante l'ottimo grado di maturità raggiunto dal prototipo, sono state individuate diverse direttrici per l'evoluzione del sistema:

1. **Compressione Dati e Lifetime Extension:** Implementazione di algoritmi di compressione *on-the-fly* (es. LZ4 o delta-encoding per le coordinate) per ridurre il numero di scritture fisiche sulla Flash, estendendo la vita utile del chip ben oltre i 10^5 cicli nominali.
2. **Sincronizzazione Temporale Avanzata:** Utilizzo del segnale *Timepulse* (PPS) del modulo GNSS per disciplinare l'oscillatore interno dell'RP2040, permettendo data-logging con precisione temporale al microsecondo.

3. **Connettività LPWAN:** Integrazione di un ricetrasmettitore LoRaWAN per l'invio di pacchetti di telemetria sintetizzati, trasformando il logger in un nodo IoT attivo per il monitoraggio remoto.
4. **Power Management Predittivo:** Implementazione di una logica di *Dynamic Voltage and Frequency Scaling* (DVFS) basata sullo stato di carica della batteria e sulla frequenza di acquisizione richiesta.

In conclusione, il sistema sviluppato non rappresenta solo un esercizio di stile accademico, ma una solida base per lo sviluppo di unità di monitoraggio ambientale affidabili, modulari e pronte per un'eventuale ingegnerizzazione di pre-serie.

A Documentazione del Codice Sorgente

In questa appendice viene riportata l'organizzazione modulare del firmware sviluppato per il SoC RP2040. Il codice è stato ingegnerizzato seguendo criteri di modularità e astrazione dell'hardware, per facilitare la manutenzione e il testing dei singoli sottosistemi.

A.1 Struttura del Repository

La gerarchia del progetto rispecchia la separazione tra configurazione dell'ambiente di build (gestita da PlatformIO), codice sorgente principale e moduli driver specifici per le periferiche.

```
.
|-- Makefile             # Script per l'automazione del processo di build
|-- platformio.ini       # Configurazione ambiente, librerie e partizioni Flash
|-- README.md            # Documentazione sintetica del repository
`-- src
    |-- main.cpp         # Entry point: inizializzazione core e loop principale
    |-- minmea           # Libreria esterna per il parsing delle sentenze NMEA
    |   |-- minmea.c
    |   `-- minmea.h
    `-- Modules          # Moduli driver e logica di astrazione hardware
        |-- buttons.cpp  # Gestione ISR e debouncing asincrono dei pulsanti
        |-- buttons.h
        |-- buzzer.cpp   # Driver per segnali acustici di feedback
        |-- buzzer.h
        |-- display_ui.cpp # Logica di rendering e interfaccia grafica OLED
        |-- display_ui.h
        |-- Functions.cpp  # Funzioni helper e utility di sistema
        |-- Functions.h
        |-- gps.cpp      # Gestione UART e interfacciamento con SAM-M8Q
        |-- gps.h
        |-- leds.cpp     # Controllo striscia LED RGB (tramite offloading PIO)
        |-- leds.h
        |-- temp.cpp     # Interfaccia per sensore di temperatura ambientale
        `-- temp.h
```

Listing 1 Albero del progetto firmware e descrizione dei moduli

A.2 Configurazione e Riproducibilità dell'Ambiente

Il file `platformio.ini` costituisce la *Single Source of Truth* del progetto. Esso garantisce che l'intero ecosistema — comprensivo di toolchain ARM GCC, framework Arduino-Pico e dipendenze esterne — sia vincolato a versioni specifiche, assicurando la determinazione dell'ambiente di build su qualsiasi postazione di lavoro.

Gestione delle Dipendenze e del Core

A differenza dell'ambiente Arduino IDE standard, PlatformIO permette di specificare il core di Earle Philhower (`earlephilhower`), ottimizzato per il SoC RP2040, e di definire rigorosamente il partizionamento della memoria Flash per ospitare il filesystem *LittleFS*.

```
[env:pico]
platform = https://github.com/maxgerhardt/platform-raspberrypi.git@1.18.0
board = nanorp2040connect
board_build.core = earlephilhower
board_build.filesystem_size = 65536 ; Riserva 64KB per LittleFS
upload_protocol = picotool
```

```
lib_deps =
    adafruit/Adafruit GFX Library @ ^1.12.4
    adafruit/Adafruit SSD1306 @ ^2.5.16
    fastled/FastLED @ ^3.10.3
```

Listing 2 Configurazione platformio.ini per l'ottimizzazione del core RP2040.

Workflow di Sviluppo e Automazione tramite Makefile

Sebbene PlatformIO offra una potente interfaccia a riga di comando (*CLI*), è stato predisposto un `Makefile` [15] per astrarre le operazioni comuni e gestire flag di compilazione condizionale. Questo permette di configurare il comportamento del firmware (es. attivazione dei log di debug o feedback acustici) senza modificare il codice sorgente.

- **Compilazione Condizionale**: Tramite la variabile `PLATFORMIO_BUILD_FLAGS`, vengono iniettate macro nel preprocessore C++ (es. `DEBUG`, `BUZZER_INIT_BEEP`).
- **Target Specializzati**: Il comando `make debug` automatizza il ciclo build-upload-monitor, riducendo i tempi morti durante la fase di testing.

```
PIO = pio
DEBUG ?= 0
BEEP ?= 1

# Iniezione dei flag nel preprocessore tramite variabili d'ambiente
FLAGS = -Os -DDEBUG=$(DEBUG) -DBUZZER_INIT_BEEP=$(BEEP)
export PLATFORMIO_BUILD_FLAGS = $(FLAGS)

.PHONY: all debug fs clean

# Target standard: Compilazione e caricamento
all:
  $(PIO) run --target upload

# Profilo Debug: Log attivi e monitoraggio seriale automatico
debug:
  $(PIO) run --target upload
  $(PIO) device monitor

# Gestione Filesystem: Caricamento dati in LittleFS
fs:
  $(PIO) run --target uploadfs

clean:
  $(PIO) run --target clean
```

Listing 3 Makefile per la gestione dei profili di build (Silent, Debug, Production).

Comandi CLI Fondamentali

Per operazioni granulari, la CLI di PlatformIO mette a disposizione i seguenti strumenti:

- `pio run`: Esegue il processo di build analizzando l'albero delle dipendenze.
- `pio run -t buildfs`: Genera l'immagine binaria del filesystem a partire dalla cartella locale `data/`.
- `pio device monitor`: Avvia l'ascolto sulla porta seriale virtuale (CDC) per il debugging real-time.

A.3 Implementazione dei Moduli Hardware

Questa sezione raccoglie le implementazioni salienti dei driver sviluppati, con particolare enfasi sulla gestione del determinismo e delle risorse del SoC RP2040.

Sottosistema di Input: buttons.cpp

Contiene l'implementazione delle Routine di Servizio degli Interrupt (ISR) e la logica di validazione temporale (*debouncing*) tramite gli Alarm del Pico-SDK, come discusso nella Sezione 5.

Implementazione della logica interrupt Per abilitare l'interrupt sui gpio usiamo la funzione *gpio_set_irq_enabled_with_calll* del pico-sdk (hardware/gpio.h)

```
pinMode(SWpin[i], INPUT_PULLUP);

gpio_set_irq_enabled_with_callback(SWpin[i], GPIO_IRQ_LEVEL_HIGH, false, &
    buttons_callback);
gpio_set_irq_enabled_with_callback(SWpin[i], GPIO_IRQ_LEVEL_LOW, false, &
    buttons_callback);
gpio_set_irq_enabled_with_callback(SWpin[i], GPIO_IRQ_EDGE_RISE, false, &
    buttons_callback);
gpio_set_irq_enabled_with_callback(SWpin[i], GPIO_IRQ_EDGE_FALL, true, &buttons_callback
    );
```

NOTA: come misura di sicurezza vengono disabilitati gli altri trigger che non ci interessano, infatti potenzialmente possono essercene diversi attivi simultaneamente.

Essendo i segnali dei pulsanti attivi bassi, il trigger dell'interrupt avviene per il Falling Edge. La logica utilizzata per l'handling degli interrupts é la seguente: una volta rilevato il fronte di discesa (*falling edge*), il processore sospende l'esecuzione del task corrente per saltare alla routine di servizio (*ISR*) denominata `buttons_callback`. Poiché l'architettura dell'RP2040 raggruppa gli eventi GPIO in un unico segnale verso il controller NVIC (*Nested Vectored Interrupt Controller*), la funzione riceve come parametri l'identificativo del pin (`gpio`) e l'evento che ha scatenato l'interrupt (`events`).

```
void buttons_callback(uint gpio, uint32_t events) {
    if(events == GPIO_IRQ_EDGE_RISE)
    {
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_RISE, false);
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_FALL, true);
        return;
    }
    switch(gpio){

        case SW_UP:
        case SW_DOWN:
        case SW_LEFT:
        case SW_RIGHT:
        {
            gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_FALL, false);
            add_alarm_in_ms (DEBOUNCE_DELAY, &debounce_callback, (void*) gpio, true);
        } break;
        default:
        {
            ;
        }
    } return;
}
```

All'interno dell'handler, viene implementata una struttura di controllo (switch case) per discriminare la natura dell'evento; per ora si prega di ignorare il primo costrutto if, in quanto la sua utilità sarà piú chiara

successivamente. Lo switch case non discrimina il singolo pulsante fra i quattro, ma si occupa di attivare un alarm interrupt, ossia un delay non bloccante per il processore che attiva una ISR (debounce_callback) dopo un certo delay (=DEBOUNCE_DELAY). É importante notare che viene anche disabilitato l'interrupt sul falling edge, per evitare che un bounce del segnale possa generare molti interrupts.

Analizziamo ora l'handling dell'alarm interrupt:

```
long long debounce_callback(alarm_id_t id, void *user_data){
    uint PinNum=(uint) user_data;

    if(!gpio_get( PinNum ))
    {
        switch( PinNum )
        {
            case SW_UP: up_press=true; break;

            case SW_DOWN: down_press=true; break;

            case SW_LEFT: left_press=true; break;

            case SW_RIGHT: right_press=true; break;

            default:
            {
                break;
            }
        }
            gpio_set_irq_enabled(PinNum, GPIO_IRQ_EDGE_RISE, true);
            return 0;
    }
    gpio_set_irq_enabled(PinNum, GPIO_IRQ_EDGE_FALL, true);
    return 0;
}
```

In questo caso viene verificato (tramite *gpio_get(PinNum)*) se il Pin dopo il delay *DEBOUNCE_DELAY* é ancora basso:

- se ancora basso allora si determina il gpio dell'interrupt (variabile *user data* passata dalla funzione *buttons_callback*), e scrive nella variabile bool true. Ora viene attivato l'interrupt per il fronte di salita, per identificare il rilascio del pulsante. L'handler di tale fronte é ancora *buttons_callback*.
- se il pulsante é stato rilasciato in un tempo inferiore a *DEBOUNCE_DELAY*, allora viene riattivato l'interrupt per il fronte di discesa senza fare nient'altro.

Ritornando al primo if-case della funzione *buttons_callback*:

```
void buttons_callback(uint gpio, uint32_t events) {
    if(events == GPIO_IRQ_EDGE_RISE)
    {
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_RISE, false);
        gpio_set_irq_enabled(gpio, GPIO_IRQ_EDGE_FALL, true);
        return;
    }
    ...
```

Serve per identificare il primo fronte di salita (e quindi il rilascio) dopo una pressione **valida** dei pulsanti; infatti viene disattivato l'interrupt per il fronte di salita e attivato quello per il fronte di discesa, senza fare nient'altro.

Questa scelta implementativa é stata presa per evitare che qualche rumore dato dal rilascio del pulsante possa attivare un'interrupt che causerebbe solo una falsa pressione, nel caso sfortunato in cui il rumore perdurasse per un tempo *DEBOUNCE_DELAY*.

Il main loop verifica in maniera sincrona la pressione dei pulsanti tramite le variabili bool che ne indicano la avvenuta pressione. In questo modo separiamo le decisioni software dovute dalla pressione dei pulsanti (esecuzione lenta) da quella di verifica della pressione degli stessi (esecuzione rapida).

Queste scelte mantengono il *Core* libero da cicli di *polling* intensivi, delegando la logica di debouncing e l'attesa di un evento direttamente all'hardware del SoC. Tuttavia é necessaria dell'attenzione aggiuntiva per gestire situazioni in cui gli interrupt generati potrebbero interrompere fasi critiche del programma, ad esempio il flusso di dati proveniente dal modulo GNSS SAM-M8Q o lo scambio dati con il PC.

Interfaccia GNSS: gps.cpp

Implementa la ricezione seriale asincrona e il parsing deterministico delle sentenze NMEA (`$GPRMC`, `$GPGGA`) tramite la libreria *minmea*.

PARSING STRINGHE NMEA

```
// Esempio di parsing di una sentenza RMC
if (minmea_parse_rmc(&frame, line)) {
    float lat = minmea_tocoord(&frame.latitude);
    float lon = minmea_tocoord(&frame.longitude);
    float speed = minmea_tofloat(&frame.speed);
}
```

Schermo

```
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Inizializzazione con indirizzo I2C
if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("Errore allocazione SSD1306"));
}
display.clearDisplay(); // Svuota il buffer interno
```

LED

```
#include <FastLED.h>
CRGB leds[NUM_LEDS];

// Impostazione colore tramite spazio HSV (Tonalita, Saturazione, Valore)
leds[0] = CHSV(hue, 255, 255);
FastLED.show();
```

File System

Per importare la libreria é sufficiente

```
#include "LittleFS.h"
```

e per montare il filesystem:

```
LittleFS.begin();
```

Di default se il mount fallisce la libreria formatta la parte di flash riservata al filesystem e riprova il mount.

Altre funzioni utilizzate sono:

```
//Apre un file in maniera simile
//ad fopen (ritorna stream pointer)
File f = LittleFS.open(path, mode);

//Rimuove un file
//(ritorna bool)
LittleFS.remove(path);

//Verifica se il path esiste
//(ritorna bool)
LittleFS.exists(path);

//Crea una cartella
//(ritorna bool)
LittleFS.mkdir(path);

//Rimuove una cartella
//(ritorna bool)
LittleFS.rmdir(path);
```

CAPIRE DOVE METERE: Il firmware interagisce con il PIO tramite una semplice interfaccia di scrittura:

```
// Caricamento del dato nel buffer FIFO della State Machine
pio_sm_put_blocking(pio0, sm, color_data << 8u);
```

B Progetto Hardware e Documentazione Tecnica

In questa appendice vengono sintetizzati brevemente i dettagli costruttivi del sistema, includendo gli schemi elettrici principali, le scelte relative al layout del PCB e la distinta dei componenti (BOM).

B.1 Schemi Elettrici

Il design è stato suddiviso in blocchi funzionali per garantire la modularità e facilitare il debugging. Di seguito vengono riportati i circuiti principali:

Sottosistema di Elaborazione e Memoria

Include il SoC Raspberry Pi RP2040, il circuito di clock a 12 MHz, il pulsante di *Reset* e la memoria Flash W25Q16 collegata tramite bus QSPI.

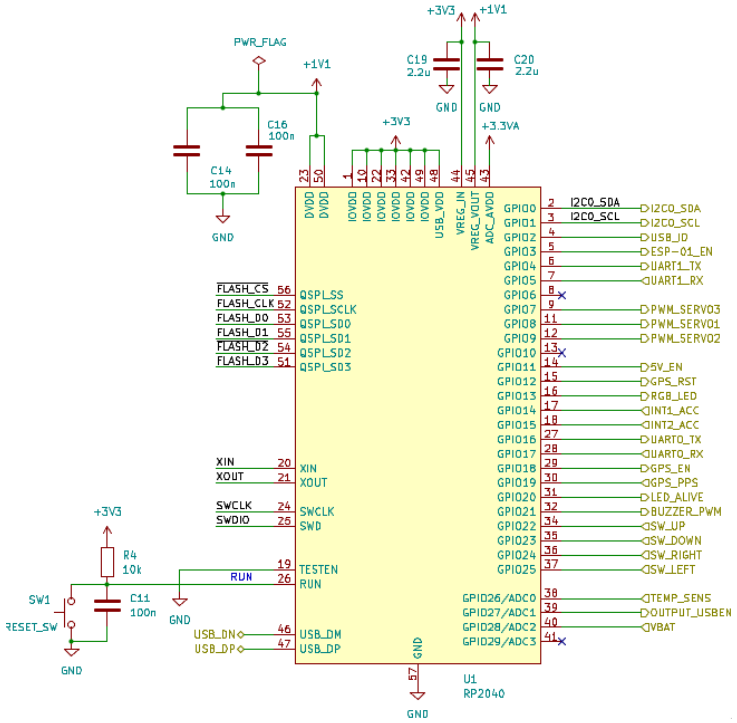

Figura 2 Schema elettrico del core RP2040 e periferiche di supporto.

Sottosistema GNSS e RF

Circuito di interfaccia del modulo u-blox SAM-M8Q, comprensivo di filtraggio dell'alimentazione dedicato e connessione UART verso l'MCU.

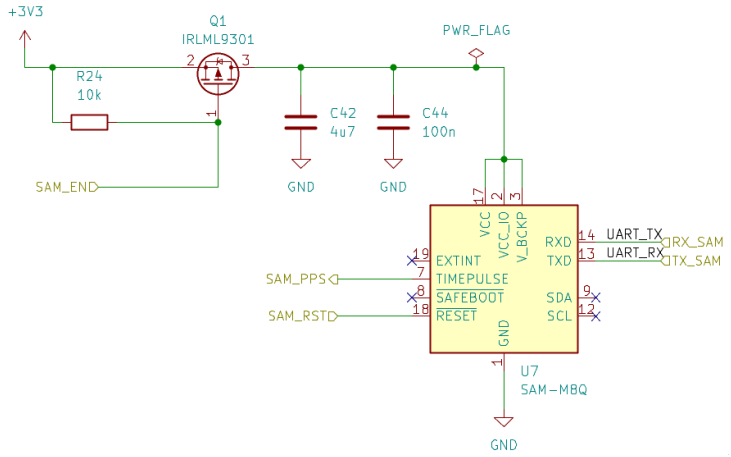

Figura 3 Schema del modulo GNSS e sezione antenna.

Buck converter

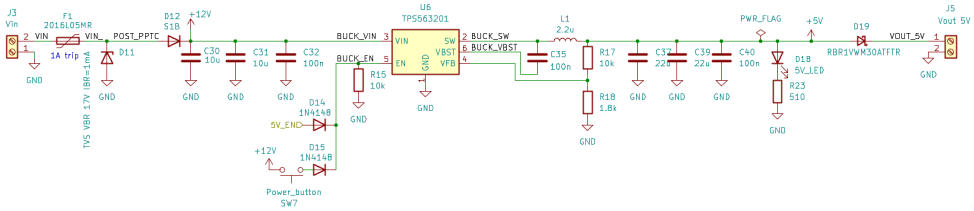

Figura 4 Schema circuitale del convertitore DC-DC buck per la linea di potenza a 5 V.

LDO

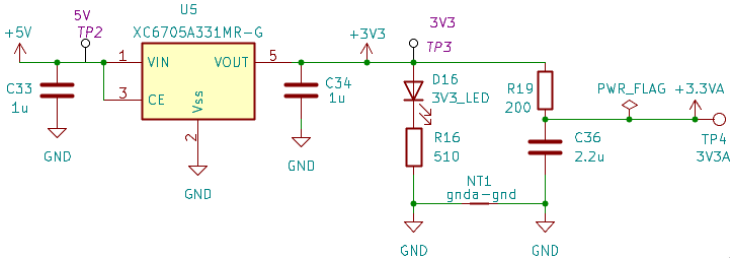

Figura 5 Stadio di regolazione lineare per l'alimentazione della logica e del modulo GNSS.

Pulsanti

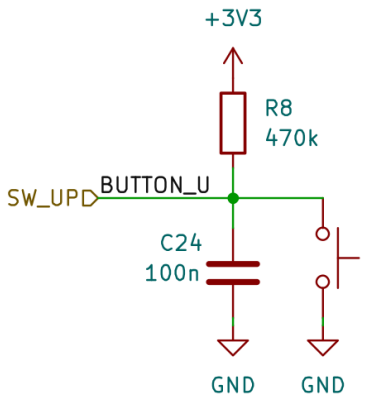

Figura 6 Schematico dei pulsanti: rete RC e Pull-up interno.

Flash

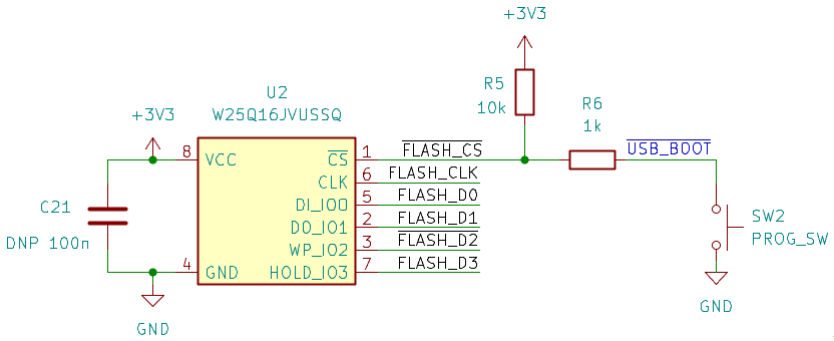

Figura 7 Circuito di interfaccia QSPI tra RP2040 e memoria Flash [12].

Riferimenti bibliografici

[1] u-blox AG. *SAM-M8Q u-blox M8 GNSS antenna module Data Sheet*, 2020. Rev. R06.

[2] Raspberry Pi Ltd. *RP2040 Datasheet: A microcontroller by Raspberry Pi*, 2023. Rev. 2.1. Disponibile online: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf.

[3] Eben Upton and James Adams. The rp2040: A high-performance microcontroller with programmable i/o. In *2021 IEEE Hot Chips 33 Symposium (HCS)*, pages 1–23. IEEE, 2021.

[4] Texas Instruments. *TPS563201: 3-A Synchronous Step-Down Voltage Regulator*, 2016.

[5] Raspberry Pi Ltd. *Raspberry Pi Pico Datasheet: An RP2040-based microcontroller board*, 2022. Rev. 1.9.

[6] Ivan Kravets. Platformio: An open source ecosystem for iot development. https://platformio.org/, 2024. Accessed: 2026-01-24.

[7] Earle F. Philhower. *Raspberry Pi Pico Arduino core, based on the official Raspberry Pi C/C++ SDK*, 2023. Version 3.0.0.

[8] Kosma Moczek. minmea: a lightweight gps parser library in pure c. https://github.com/kosma/minmea, 2021. Accessed: 2026-01-24.

[9] Adafruit Industries. Adafruit gfx graphics library. https://github.com/adafruit/Adafruit-GFX-Library, 2024. Accessed: 2026-01-24.

[10] Adafruit Industries. Adafruit ssd1306 oled driver library for arduino. https://github.com/adafruit/Adafruit_SSD1306, 2024. Accessed: 2026-01-24.

[11] Daniel Garcia and Mark Kriegsman. Fastled: A library for easily & efficiently controlling a wide variety of led chipsets. https://fastled.io/, 2024. Accessed: 2026-01-24.

[12] Winbond Electronics. *W25Q128JV: 3V 128M-Bit Serial Flash Memory with dual/quad SPI*, 2018. Rev. F.

[13] ARM Ltd. Littlefs: A high-integrity embedded file system for microcontrollers. https://github.com/littlefs-project/littlefs, 2023. Accessed: 2025-01-24.

[14] Nmea 0183: Standard for interfacing marine electronic devices. Standard, National Marine Electronics Association (NMEA), Severna Park, MD, USA, 2008. Version 4.00.

[15] Richard M. Stallman, Roland McGrath, and Paul D. Smith. *GNU Make: A Program for Directing Recompilation*. Free Software Foundation, 2023.